

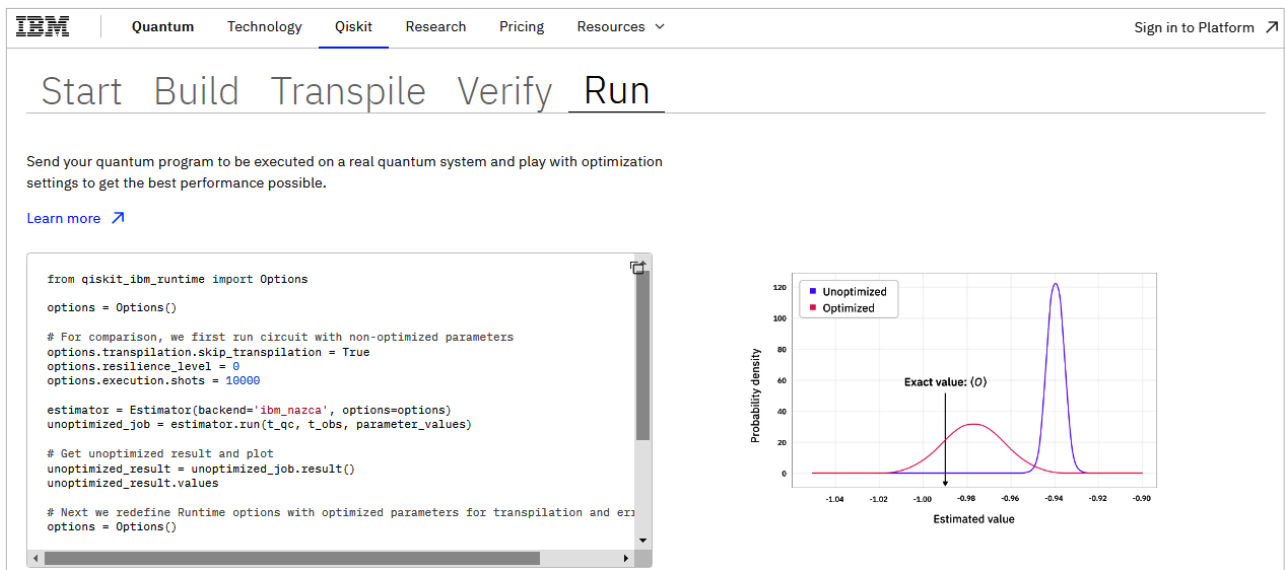


Figure 4: Start, Build, Transpile, Verify and Run in Qiskit

```
Get counts of the measurement outcomes
counts = result.get_counts()
```

```
Print the measurement outcomes
print("Measurement outcomes:", counts)
```

### Output for example 1: Qiskit (Python)

```
Measurement outcomes: {'00': 515, '11': 509}
```

This output indicates that the simulator produced approximately equal numbers of '00' and '11' outcomes, which is expected for the entangled state produced by the Hadamard and CNOT gates.

## Quantum implementation using Q (Q Sharp)

```
// Define a quantum operation to create
an entangled state
operation Entangle(qubit1 : Qubit,
qubit2 : Qubit) : Unit {
    H(qubit1);
    CNOT(qubit1, qubit2);
}
```

```
// Define a quantum operation to measure
qubits and return the result
operation MeasureEntangled(qubit1 :
```

```
Qubit, qubit2 : Qubit) : (Result,
Result) {
    let result1 = M(qubit1);
    let result2 = M(qubit2);
    return (result1, result2);
}
```

```
// Define a driver operation to run the
quantum operations
operation RunEntanglementProtocol() :
(Result, Result) {
    // Allocate two qubits
    using (qubit1 = Qubit(), qubit2 =
Qubit()) {
        // Entangle the qubits
        Entangle(qubit1, qubit2);

        // Measure the entangled qubits
        let (result1, result2) =
MeasureEntangled(qubit1, qubit2);

        // Return the measurement
results
        return (result1, result2);
    }
}
```

### Output

When `RunEntanglementProtocol()` operation is executed, it returns a tuple of measurement results `(Result, Result)`, where each `Result` represents

the outcome of measuring a qubit.

These examples demonstrate how to create simple quantum circuits, apply quantum gates, simulate measurements, and execute quantum operations using Qiskit and Q. Developers can run these code snippets in their respective environments (e.g., IBM Quantum Experience for Qiskit or Microsoft Quantum Development Kit for Q) to explore quantum computing concepts and experiment with quantum algorithms.

The scope of research in the simulation of quantum computing is vast and multifaceted.

- **Algorithm development:** Researchers can explore and develop new quantum algorithms for various computational tasks, such as optimisation, cryptography, and machine learning. By simulating these algorithms, they can analyse their performance, scalability, and potential advantages over classical counterparts.
- **Error correction and fault tolerance:** Investigating error correction techniques and fault-tolerant quantum computing is crucial for overcoming noise and decoherence in quantum systems.